

gasmet

Jake Kundert

2024-04-03

Jesse's modifications

I want coefs.sum need time and date data for processing co2.e.kghyr per day need to make this code work for multiple experiments, or run each of 5 experiments separately

need to clean up the data a bit, like experiment name

Jesse's suggestions

Definitions of coding terms co2 vs co2.c

Gasmet Data Analysis

Gasmet output provide a gas concentration value at each time step measured. This script takes that typical gasmet data, cleans it, makes conversions, and calculates a flux value from each sampling curve. If necessary for your analysis, there is also code to integrate a series of flux values to calculate a total emissions value.

An example input .csv file can be found at [here](#).

This code assumes:

1. Each batch of data being analyzed comes from the same year. If multiple years are included in the data set, changes will need to be made to account for groupings.
2. The date column should be in the year-month-date format in the .csv file.

```
#Load packages ----
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.3      v readr      2.1.4
```

```
## v forcats    1.0.0      v stringr   1.5.0
```

```
## v ggplot2    3.4.4      v tibble    3.2.1
```

```
## v lubridate  1.9.3      v tidyr     1.3.0
```

```
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(plyr)
```

```
## Warning: package 'plyr' was built under R version 4.3.2
```

```
## -----
```

```
## You have loaded plyr after dplyr - this is likely to cause problems.
```

```
## If you need functions from both plyr and dplyr, please load plyr first, then dplyr:
```

```
## library(plyr); library(dplyr)
## -----
##
## Attaching package: 'plyr'
##
## The following objects are masked from 'package:dplyr':
##
##   arrange, count, desc, failwith, id, mutate, rename, summarise,
##   summarize
##
## The following object is masked from 'package:purrr':
##
##   compact
library(stringr)
```

Load Packages and Prep Data

This script assumes the following columns should be included in the gasmet data:

site, date, plot, time, co2, n2o, ch4, celltemp

A new column, 'datetime', is created by merging date and time. And 'code' is created to allow for a unique ID for each plot/date measurement.

```
# dat.gasmet<-read.csv("Data/Gasmet/Raw Data/gasmet_2020_eddyflux.csv", na = "NA")

read.csv(
  "jesse_KGML_Dataset.xlsx - data_daily (2).csv"
) -> dat.gasmet

dat.gasmet %>%
  mutate(experiment = fct_recode(experiment,
    "oreiManure" = "orei_manure")) %>%
  mutate(site=as.factor(site),
    date = as.POSIXct(date,
      "%m/%d/%Y", tz = ""),
    datetime = as.POSIXct(paste(date, time), "%Y-%m-%d %H:%M:%S",
      tz = ""),
    code = paste(experiment, plot, date, site, sep = "_")
  ) -> dat.add
```

Calculating Running Time

We now need to use the datetime column to create a new column with the running time of each sample (ie., code). To do this, we create the ind column which creates a sequence of numbers for each code. The row with '1' is the first sample. We will use that 1 as a flag to calculate the running total, adding each difference between the current row's datetime and the previous row's datetime.

```
dat.time <- dat.add %>%
  arrange(datetime) %>%
  group_by(code) %>%
  dplyr::mutate(ind = 1:n(), #this will label the first measurement of each code with '1'
    change.t = ifelse(ind == 1, #if its the first row...
      0, #assign a 0. If not, calculate the difference in datetimes
      as.numeric(difftime(datetime, lag(datetime)))), #should be 20 or 21s
```

```
run.t = cumsum(as.numeric(ifelse(ind != 1, #the ind is not '1' (ie. 1st sample)
                                difftime(datetime, lag(datetime)), #record the sec when sample is made
                                0))), #if ind is '1', then start at 0

n = n())
```

#it appears that the for-loop difftime is having trouble differentiating minutes from seconds. so when

QA/QC

Review the data. Things to consider are:

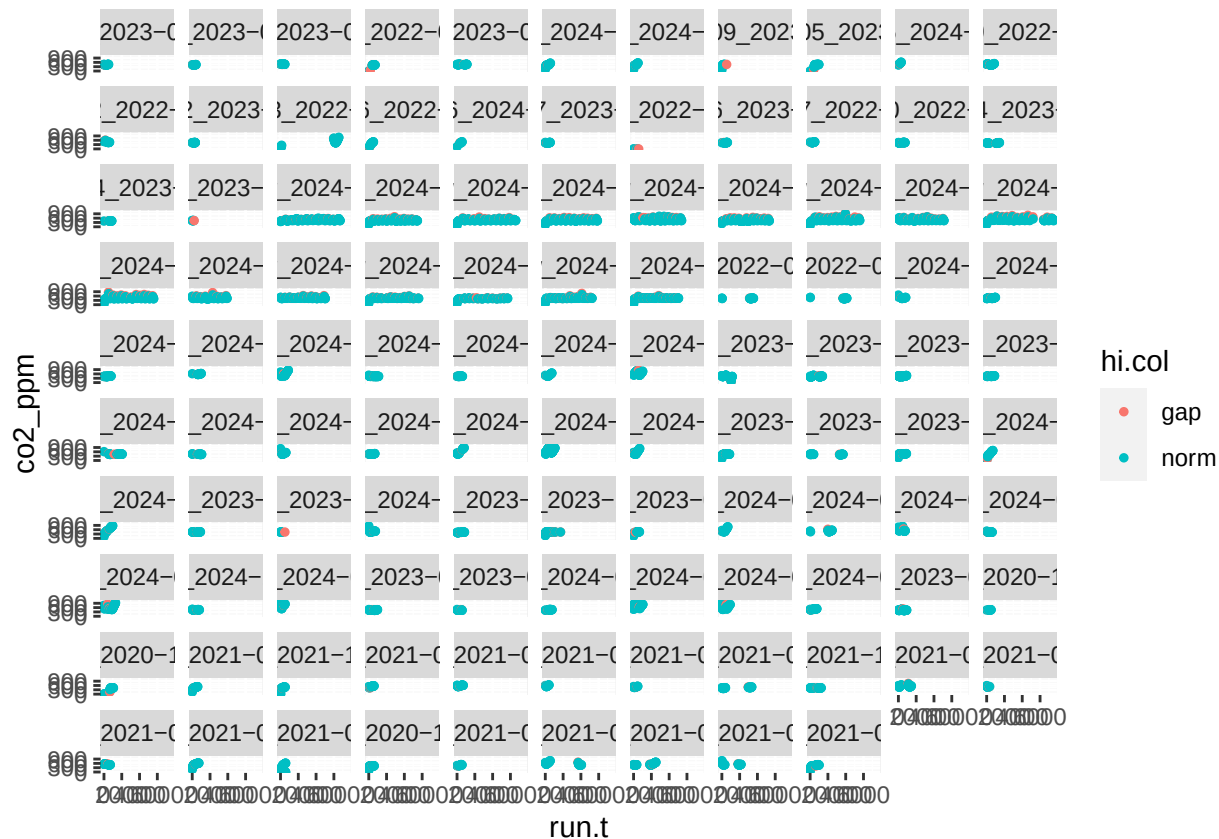
- How many unique dates are there and does this match with how many you expected?
- Was each plot sampled each day?
- How many measurements are there for each code (plot-date-site)?
- Are there gaps in the data?
- Review indicators of issues like: CO2 range, cell temp range, running time range

#create a df with codes that have a gap in sampling of >60sec

```
hi.dt<- dat.time %>%
  filter(change.t > 60)
```

#visualize the slopes of the codes that have a long gap in sampling

```
dat.time %>%
  filter(code %in% hi.dt$code) %>%
  mutate(hi.col = ifelse(change.t > 60, "gap",
                        'norm')) %>%
  ggplot(aes(run.t, co2_ppm, color = hi.col)) +
  geom_point(size = 1) +
  facet_wrap(~code)
```



```
unique(dat.time$ind) #check if any NA's exist in the ind column.
```

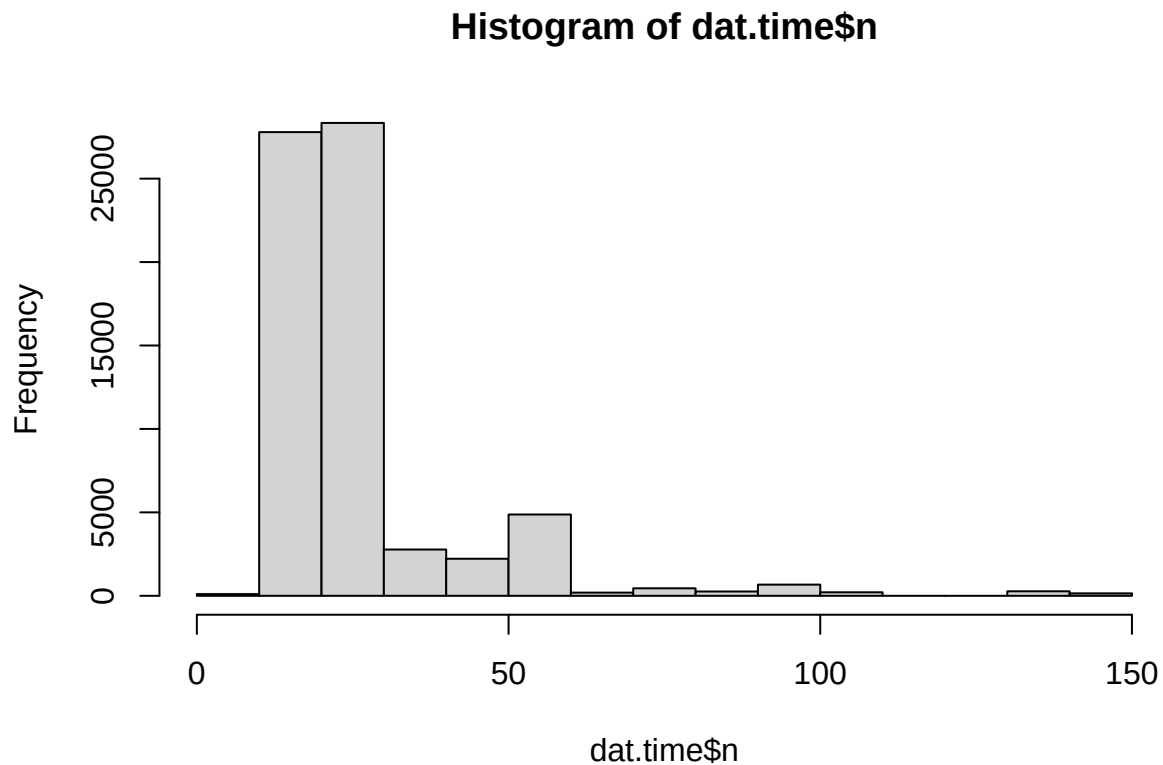
```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147
```

```
unique(dat.time$change.t) #check the values of the time change between each sequential datetime row. The
```

```
## [1] 0.0 20.0 22.0 21.0 517.0 36.0 127.0 202.0 43.0 60.0
## [11] 120.0 61.0 101.0 132.0 1487.0 1462.0 110.0 3135.0 154.0 97.0
## [21] 1.0 23.0 24.0 25.0 765.0 72.0 26.0 144.0 27.0 2554.0
## [31] 46.0 338.0 95.0 81.0 47.0 229.0 82.0 426.0 85.0 3077.0
## [41] 3670.0 89.0 39.0 107.0 194.0 59.0 5845.0 41.0 38.0 355.0
## [51] 40.0 142.0 174.0 170.0 210.0 393.0 57.0 29.0 349.0 147.0
## [61] 173.0 28.0 70.0 782.0 184.0 2904.0 51.0 187.0 53.0 242.0
## [71] 42.0 583.0 71.0 135.0 102.0 66.0 443.0 549.0 33.0 280.0
## [81] 129.0 567.0 5.0 45.0 6.0 68.0 151.0 93.0 182.0 579.0
## [91] 421.0 37.0 169.0 24.7 63.0 79.0 304.0 83.0 524.0 382.0
## [101] 297.0 118.0 179.0 35.0 593.0 275.0 317.0 420.0 372.0 329.0
## [111] 356.0 327.0 335.0 413.0 328.0 323.0 342.0 345.0 325.0 377.0
## [121] 341.0 334.0 333.0 336.0 55.0 197.0 552.0 94.0 365.0 414.0
```

```
## [131] 494.0 462.0 330.0 354.0 332.0 337.0 331.0 339.0 326.0 324.0
## [141] 353.0 282.0 1954.0 371.0 138.0 75.0 340.0 314.0 435.0 315.0
## [151] 311.0 305.0 360.0 351.0 352.0 322.0 358.0 152.0 106.0 91.0
## [161] 204.0 44.0 222.0 54.0 279.0 366.0 320.0 343.0 381.0 422.0
## [171] 273.0 183.0 192.0 58.0 181.0 386.0 357.0 363.0 347.0 1173.0
## [181] 310.0 234.0 319.0 507.0 119.0 397.0 388.0 396.0 367.0 137.0
## [191] 455.0 554.0 499.0 56.0 318.0 440.0 359.0 50.0 411.0 295.0
## [201] 350.0 65.0 301.0 309.0 296.0 419.0 299.0 306.0
```

```
hist(dat.time$n) #view a histogram of the number of sample points per code
```



```
#review key metrics from the dataset
qc <- data.frame(ind.ct = length(which(dat.time$ind == 1)),
  code.ct = length(unique(dat.time$code)),
  date.ct = length(unique(dat.time$date)),
  max.co2 = max(dat.time$co2),
  min.co2 = min(dat.time$co2),
  max.temp = max(dat.time$celltemp),
  min.temp = min(dat.time$celltemp),
  max.time = max(dat.time$run.t),
  max.n = max(dat.time$n),
  min.n = min(dat.time$n))
```

```
## Warning: Unknown or uninitialised column: `co2`.
```

```
## Warning in max(dat.time$co2): no non-missing arguments to max; returning -Inf
```

```
## Warning: Unknown or uninitialised column: `co2`.
```

```
## Warning in min(dat.time$co2): no non-missing arguments to min; returning Inf
## Warning: Unknown or uninitialised column: `celltemp`.
## Warning in max(dat.time$celltemp): no non-missing arguments to max; returning
## -Inf
## Warning: Unknown or uninitialised column: `celltemp`.
## Warning in min(dat.time$celltemp): no non-missing arguments to min; returning
## Inf
qc
```

```
##   ind.ct code.ct date.ct max.co2 min.co2 max.temp min.temp max.time max.n min.n
## 1   2972    2972    149   -Inf     Inf     -Inf     Inf     7527   147     1
```

Removing Data

Based on the results of the QA/QC step, there may be data that should be removed. A couple options:

- If there are gaps in the data, all data points prior to that gap can be removed to create the `dat.nogap` dataframe. This usually occurs when the external battery dies or the computer shuts down and there is a delay to start sampling again.
- If there are specific problematic codes, they can be removed to create the `dat.remove` dataframe. You may also choose to go back to the original raw data and remove those rows at the source, then re-run data.

```
#cut off all datapoints prior to gap
dat.nogap <- dat.time %>%
  group_by(code) %>%
  dplyr::mutate(gap = ifelse(change.t > 60,
                           'y',
                           'n')) %>%
  slice(which.max(gap == 'y') : n()) %>%
  select(-gap)

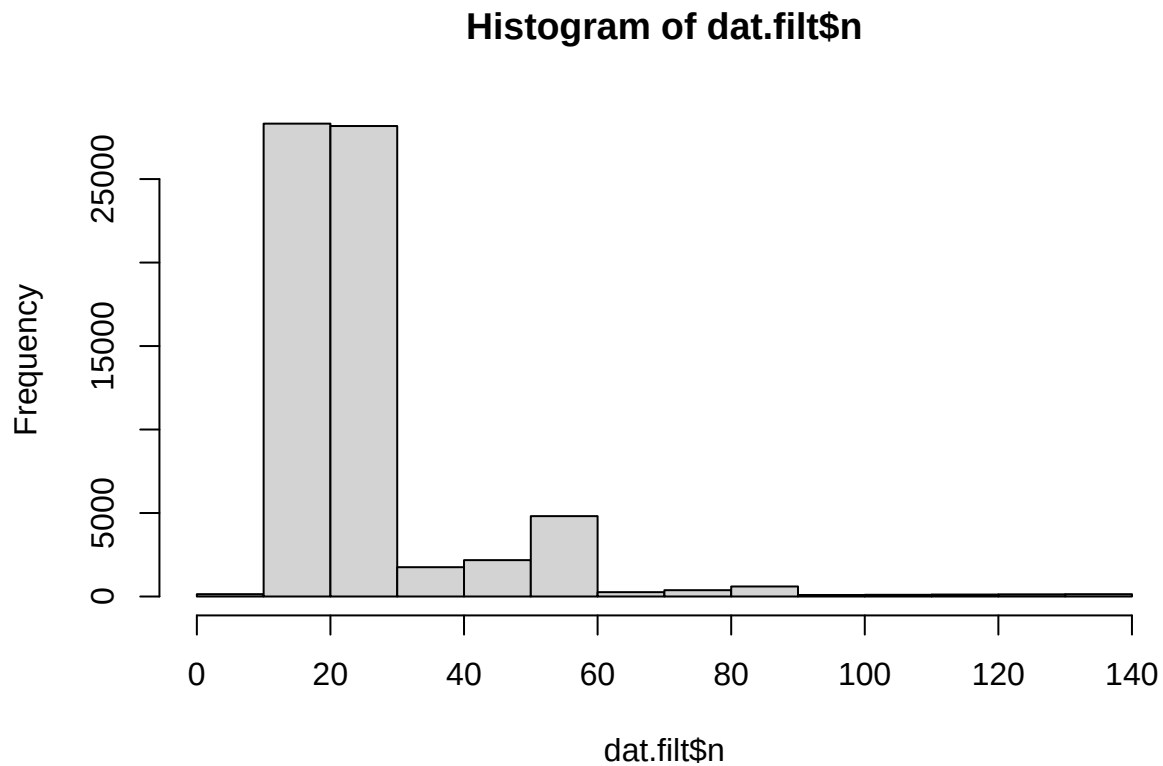
#dat.remove <- dat.time %>%
# filter(code != '8_2023-10-13_v08')
```

Rename Dataframe

Whatever dataframe you want to take to the next step, re-calculate `run.t` now that any gaps and erroneous data have been removed and rename it `dat.filt`. Then, filter out codes with 6 or fewer data points, which is required by the loess function that will be applied in the next step.

```
dat.filt <- dat.nogap %>% #dataframe with cleaned data
  arrange(datetime) %>%
  group_by(code) %>%
  dplyr::mutate(ind = 1:n(), #this will label the first measurement of each code with '1'
               change.t = ifelse(ind == 1, #if its the first row...
                                0, #assign a 0. If not, calculate the difference in datetimes
                                as.numeric(difftime(datetime, lag(datetime)))), #should be 20 or 21s
               run.t = cumsum(as.numeric(ifelse(ind != 1, #the ind is not '1' (ie. 1st sample)
                                                difftime(datetime, lag(datetime)), #record the sec when sample is made
                                                0))),
               n = n()) #if ind is '1', then start at 0
```

```
hist(dat.filt$n)
```



```
unique(dat.filt$change.t)
```

```
## [1] 0.0 21.0 22.0 20.0 36.0 43.0 60.0 110.0 1.0 23.0
## [11] 24.0 25.0 26.0 27.0 81.0 47.0 85.0 39.0 59.0 41.0
## [21] 40.0 142.0 210.0 57.0 29.0 173.0 28.0 51.0 53.0 42.0
## [31] 33.0 93.0 182.0 579.0 24.7 382.0 297.0 118.0 35.0 372.0
## [41] 338.0 329.0 356.0 327.0 335.0 413.0 328.0 323.0 342.0 325.0
## [51] 377.0 45.0 341.0 334.0 333.0 336.0 55.0 94.0 414.0 494.0
## [61] 462.0 330.0 354.0 332.0 337.0 331.0 349.0 339.0 355.0 326.0
## [71] 324.0 353.0 371.0 138.0 75.0 340.0 314.0 435.0 315.0 311.0
## [81] 360.0 351.0 352.0 322.0 358.0 91.0 204.0 44.0 317.0 320.0
## [91] 343.0 381.0 422.0 58.0 357.0 363.0 347.0 1173.0 304.0 310.0
## [101] 507.0 397.0 388.0 396.0 319.0 367.0 137.0 455.0 554.0 499.0
## [111] 549.0 56.0 345.0 318.0 440.0 50.0 295.0 350.0 65.0 301.0
## [121] 309.0 305.0 296.0 89.0 299.0 306.0
```

```
range(dat.filt$run.t) #use this range to inform numbers for 'predict.res' in next step
```

```
## [1] 0 6995
```

```
#remove any codes that have 6 or fewer data points, which is required by the loess function next
```

```
dat.filt.out <- dat.filt %>%
```

```
  filter(n <= 6)
```

```
dat.filt.in <- dat.filt %>%
```

```
filter(n > 6)
```

Creating Column to Organize and Group Data

Now to winnow down the data to just the points that we want for calculating fluxes. The goal is to get start our flux measurements ~1 minute after the chamber was placed on the soil. This process removes data points that are collecting while the machine is returning to ambient conditions between plots, the first few points once the chamber is placed which can be poor quality while the chamber is stabilizing on the soil, and creates a cut-off at the end of sampling to avoid plateauing in the data.

We will use a loess function to predict gas concentrations at a higher resolution than our 20s data points. Then find the point when the gas concentration is lowest and use that as our assumed point when the chamber was placed on the soil.

```
#create a df with a column of desired resolution to predict using loess function. Number of seconds and
predict.res <- data.frame(run.t = seq(0, 600, 0.1))

nest<- dat.filt.in %>%
  # filter(!is.na(co2_ppm)) %>%
  mutate(co2=co2_ppm) %>%
  group_by(code) %>% #create nested dfs grouped by code
  nest() %>%
  dplyr::mutate(loess.mod = map(data, ~stats::loess(co2 ~ run.t, degree = 2, data=.x)),
    #create loess functions for each code
    pred = map(loess.mod, ~stats::predict(.x, predict.res)),
    #applying loess function to predict run.t values at the resolution described in predict.res
    pred.df = map(pred, ~data.frame(co2 = .x, pred.run.t = predict.res$run.t)),
    #a new df with just the high res run.t values and predicted co2
    min.time = map(pred.df, ~slice_min(.data = .x, co2, n = 1))) %>%
  #returns the row with the minimized co2
  unnest(min.time) %>%
  dplyr::mutate(clean.data = map2(.x = data, .y = pred.run.t, ~filter(.x, run.t >= .y)))
#uses the identified minimum run.t to filter out data prior to the minimum co2 point in a cleaned df

#unnests the cleaned data and makes new count column to help remove additional points
dat.clean <- nest %>%
  select(code, clean.data) %>%
  unnest(clean.data) %>%
  dplyr::mutate(count = 1:n())

cut.off<- nest %>%
  select(code, co2, pred.run.t)
```

Plots of Pre- and Post- Cleaned Data

Plot 1: Uncleaned data.

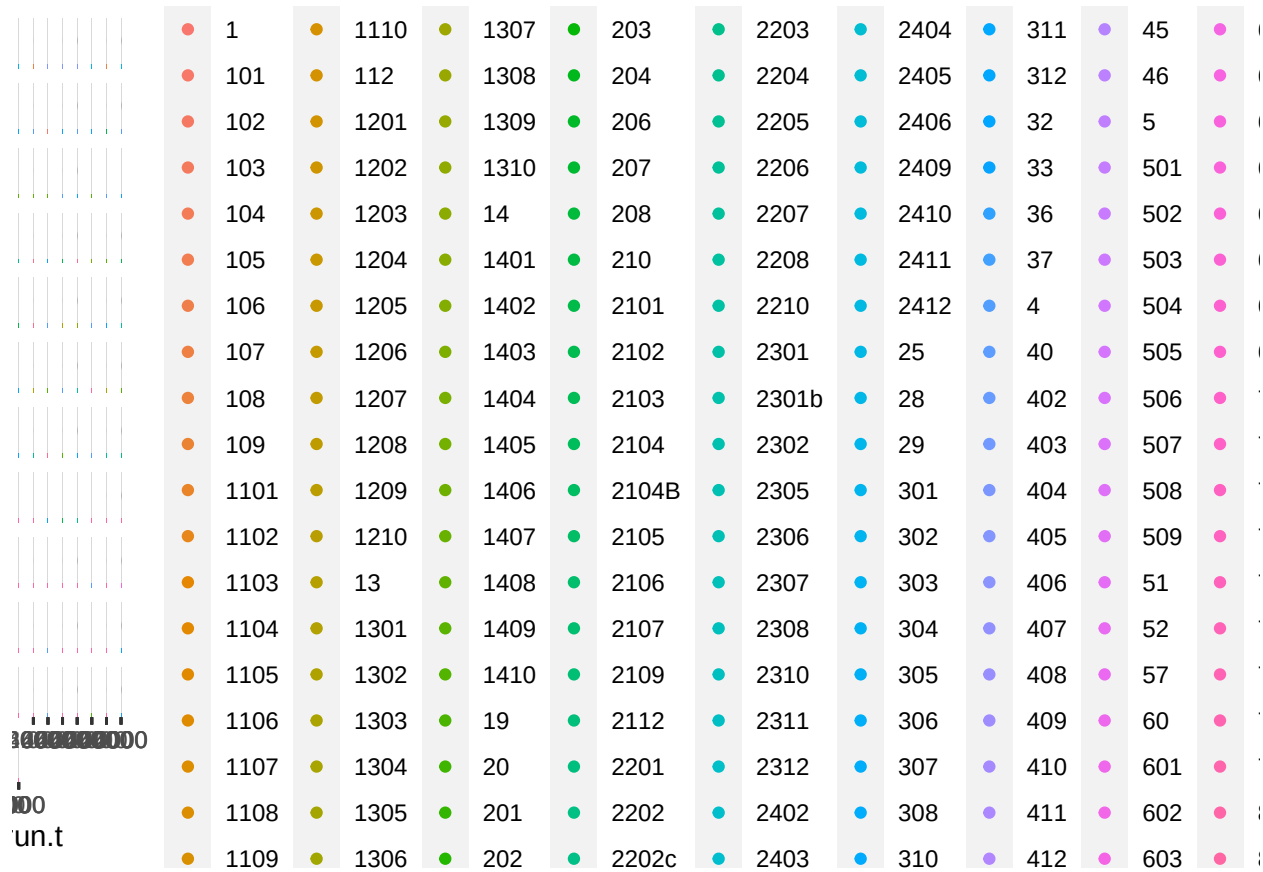
Plot 2: Cleaned data. Instead of using run.t on the x axis, this plot uses the new 'count' column which allows the plots to all start at the same point.

Plot 3: Tests to see what would be good cut off points at the beginning and end of the data to avoid poor data quality while chamber stabilizes and avoids plateauing at end.

```
#plot of pre-cleaned data
dat.filt %>%
  ggplot(aes(run.t, co2_ppm, color = as.factor(plot))) +
```

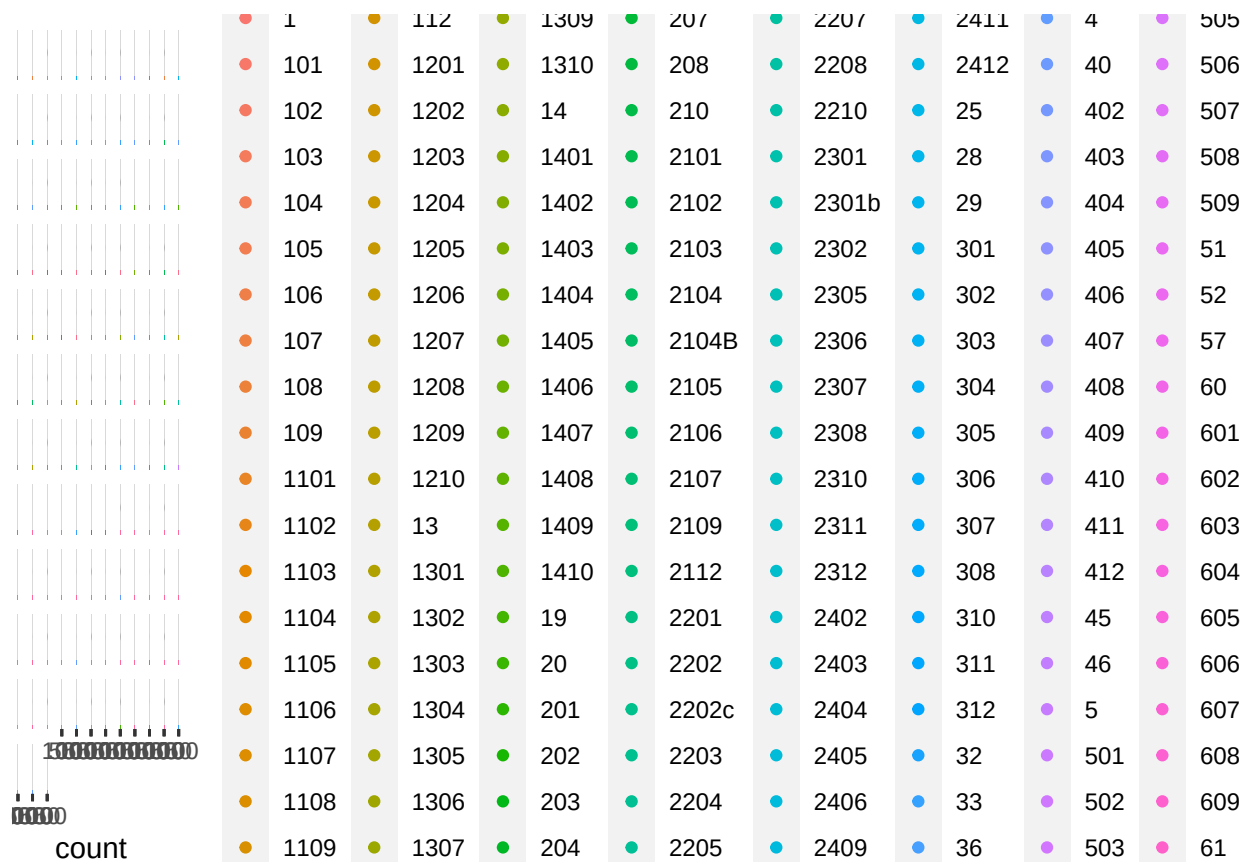


```
geom_point() +
facet_wrap(~date)
```



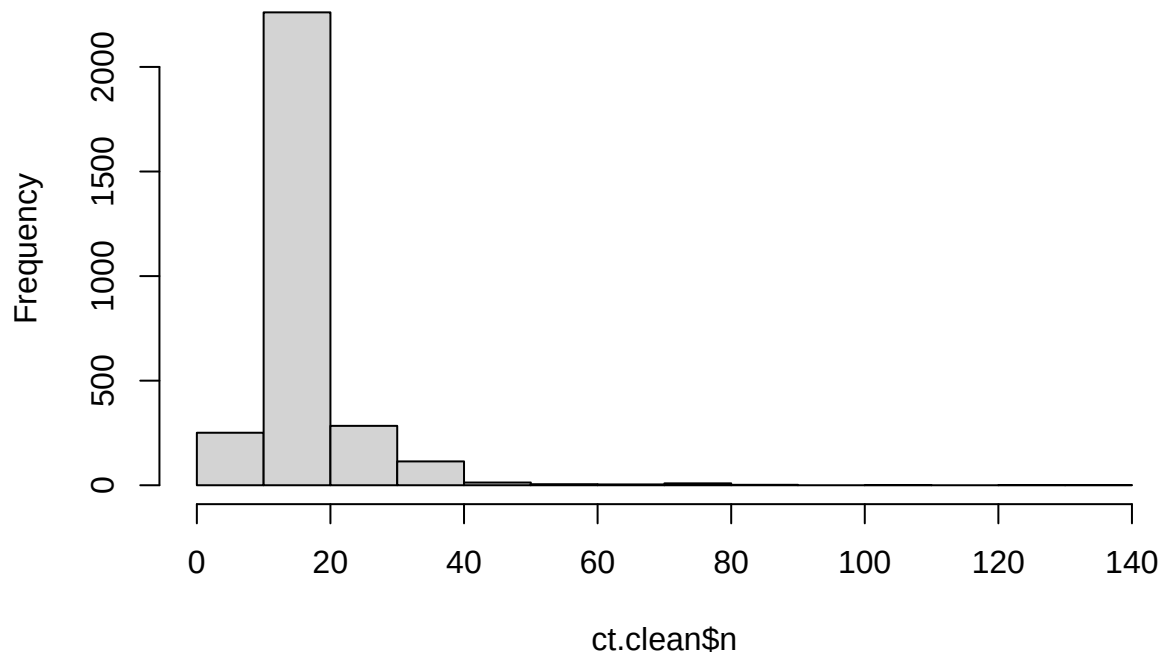
#plot of post-cleaned data. NOTICE: this uses 'count' on the x-axis so that all points align, whereas r
dat.clean %>%

```
ggplot(aes(count, co2, color = as.factor(plot))) +
geom_point() +
facet_wrap(~date)
```



```
ct.clean <- dat.clean %>%
  group_by(code) %>%
  tally()
hist(ct.clean$n)
```

Histogram of ct.clean\$n



```
ct.clean %>%
  filter(n < 10)
```

```
## # A tibble: 136 x 2
##   code          n
##   <chr>        <int>
## 1 ATAT_1107_2022-10-27_r100    9
## 2 ATAT_1307_2022-10-27_r100    5
## 3 ATAT_1406_2022-10-27_r100    9
## 4 atat_1101_2023-05-31_r100    8
## 5 atat_1101_2023-06-27_r100    9
## 6 atat_1108_2023-08-07_r100    8
## 7 atat_1108_2023-08-21_r100    6
## 8 atat_1205_2023-08-21_r100    9
## 9 atat_1405_2023-05-16_r100    1
## 10 atat_1409_2023-06-28_r100    7
## # i 126 more rows
```

```
dat.clean %>%
  filter(count > 3 & count < 10) %>% #6 data points would be 2 minute of data
  ggplot(aes(count, co2, color = as.factor(plot))) +
  geom_point() +
  geom_line() +
  facet_wrap(~date)
```



```
dat.clean %>%
  filter(count > 3 & count < 10) %>%
  ggplot(aes(count, co2, color = as.factor(date))) +
  geom_point() +
  geom_line() +
  facet_wrap(~plot)
```

2020-11-02		2022-06-01		2022-10-11		2023-06-06		2023-07-25		2023-10-20	
2020-11-05		2022-06-07		2022-10-21		2023-06-09		2023-08-01		2023-10-23	
2020-11-19		2022-06-08		2022-10-27		2023-06-14		2023-08-02		2023-10-27	
2021-04-05		2022-06-16		2023-04-13		2023-06-15		2023-08-04		2023-11-07	
2021-04-22		2022-06-28		2023-04-14		2023-06-19		2023-08-07		2024-03-13	
2021-04-29		2022-07-01		2023-04-25		2023-06-20		2023-08-08		2024-03-19	
2021-05-18		2022-07-14		2023-04-26		2023-06-21		2023-08-15		2024-04-04	
2021-06-01		2022-07-21		2023-05-08		2023-06-27		2023-08-16		2024-04-09	
2021-06-15		2022-08-01		2023-05-09		2023-06-28		2023-08-17		2024-04-15	
2021-06-21		2022-08-03		2023-05-10		2023-07-03		2023-08-21		2024-04-22	
2021-07-08		2022-08-31		2023-05-16		2023-07-04		2023-08-30		2024-04-25	
2021-07-16		2022-09-02		2023-05-17		2023-07-05		2023-09-12		2024-05-01	
2021-08-10		2022-09-08		2023-05-22		2023-07-06		2023-09-13		2024-05-03	
2021-09-10		2022-09-15		2023-05-23		2023-07-10		2023-09-14		2024-05-06	
2021-10-14		2022-09-19		2023-05-24		2023-07-11		2023-09-26		2024-05-09	
2021-10-25		2022-09-21		2023-05-25		2023-07-17		2023-09-27		2024-05-22	
2022-05-04		2022-09-27		2023-05-31		2023-07-19		2023-10-02		2024-05-23	
2022-05-05		2022-10-03		2023-06-01		2023-07-20		2023-10-09		2024-05-29	
2022-05-16		2022-10-07		2023-06-05		2023-07-24		2023-10-18		2024-05-30	

Selecting data to calculate fluxes on

Based on the plots from the previous chunk, determine a range of data to pass on to do flux calculations. Use ‘count’ to remove data at the beginning (during dead band period) and at end (potential for plateauing of data). Then calculate the number of data point each code has under that ‘count’ filter. Finally, filter out any codes that do not have enough data to calculate an accurate flux.

```
dat.slim <- dat.clean %>%
  filter(count > 3 & count < 10) %>%
  dplyr::mutate(n = n()) %>%
  filter(n > 3) %>%
  ungroup()
```

Refining and Converting

Create a function to convert co2 concentrations (ppm CO2) to ug C/cm3.

Based on Rod Venterea’s spreadsheet, which can be cited as:

Venterea RT. Simplified method for quantifying theoretical underestimation of chamber-based trace gas fluxes. *J Environ Qual.* 2010; **39**: 126–135.

The equation is $=(\text{element mass}/\text{ambient pressure}) * (1/1000000) * (1/0.082057) * (1/273.15 + \text{chamber temp}) * 1000$

Then create a new dataframe, which selects a subset of columns, creates a unique ID column ‘code’, and uses the created function to convert to co2.c using co2 and cell temp columns.

Then, save this intermediate data in a relevant folder on your computer.

```
co2.convert <- function(co2, temp) {
  co2*(12/1)*(1/1000000)*(1/0.082057)*(1/(273.15 + temp)) * 1000
}

mydat<- dat.slim %>%
  dplyr::select(code, date, plot, experiment, site, datetime, co2_ppm, run.t, temp_celsius, count) %>%
  dplyr::mutate(co2.c = co2.convert(co2_ppm, temp_celsius))
```

Calculating Slopes

Create a list of regressions for every ‘code’ (i.e., plot/date/field combination). Then create a dataframe (“coefs”) that includes the intercepts and slopes of each model. Then, separate “code” column, so plot, date, and site are in separate columns. Rename columns. Store this data in a relevant folder.

```
regressions <- dlply(mydat, .(code), lm, formula = co2.c ~ run.t)

coefs <- cbind(ldply(regressions, coef), #extracts the code variables of each model, and the 5th coefficient
  ldply(regressions, function(x){coef(summary(x))[2]}[,2:3]) #extracts the standard error

str_split_fixed(coefs$code, "_", n=3)[,1] -> coefs$experiment
str_split_fixed(coefs$code, "_", n=3)[,2] -> coefs$plot
str_split_fixed(coefs$code, "_", n=4)[,3] -> coefs$date
str_split_fixed(coefs$code, "_", n=4)[,4] -> coefs$site

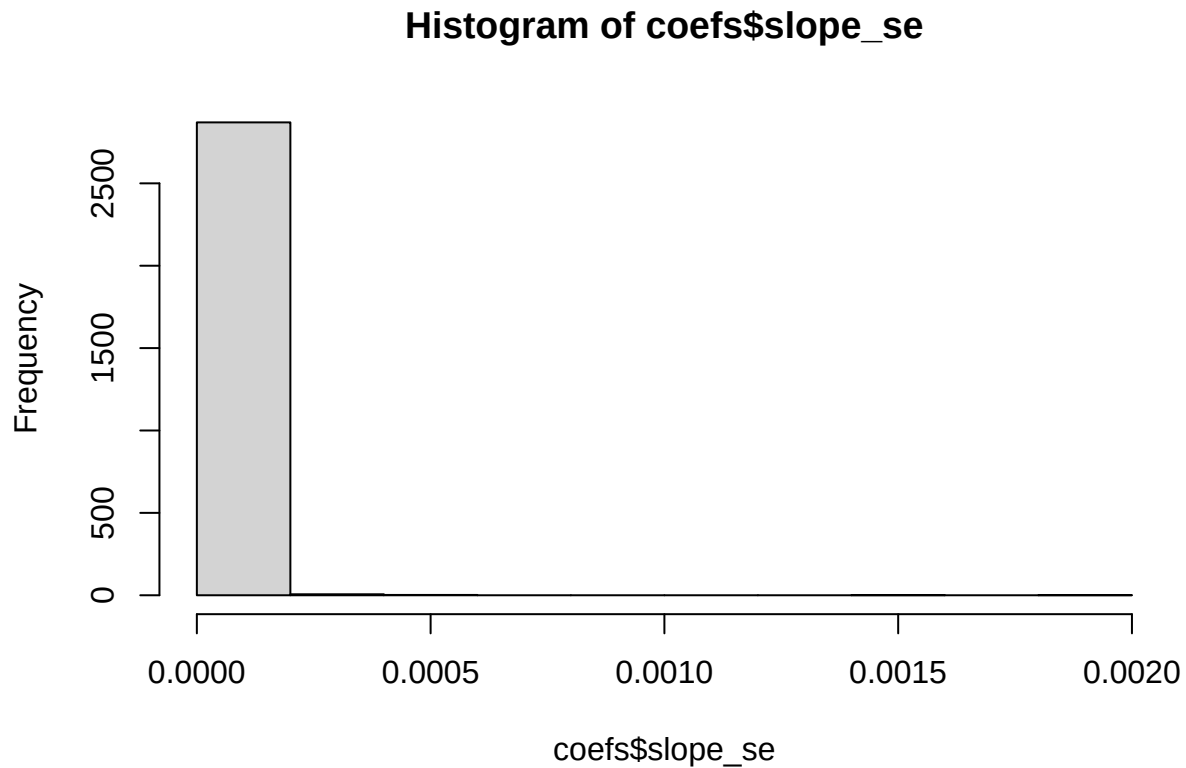
colnames(coefs)<-c("code", "intercept", "slope", "intercept_se", "slope_se", "experiment", "plot", "date", "site")
```

OPTIONAL: Remove low-quality slopes based on standard error

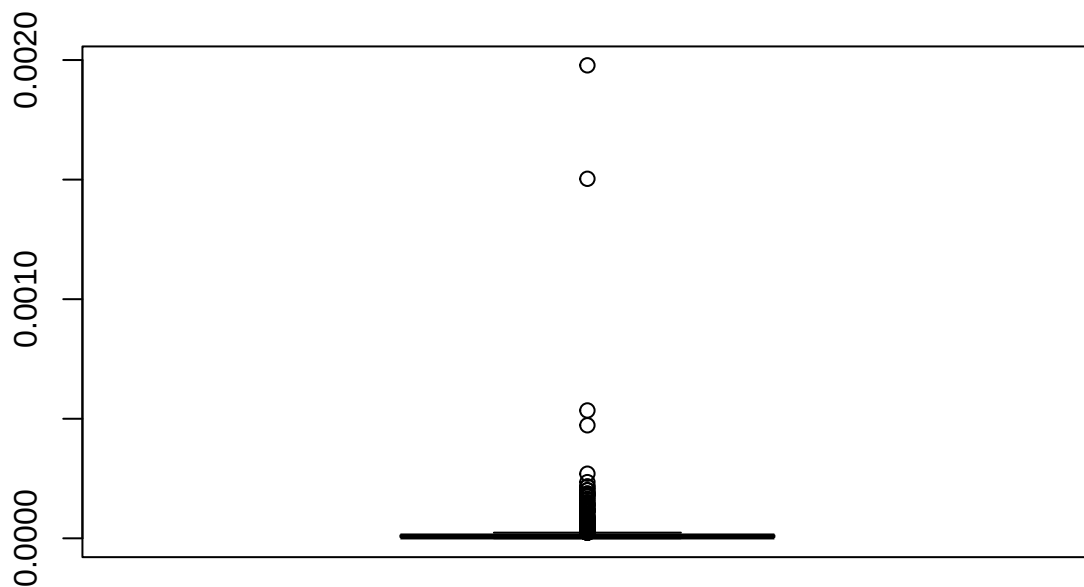
In order to remove poor quality data, the slope standard error could be a criteria for removal. Typically, poor data comes from an erratic high/low concentration measurement that falls beyond the assumed linear increase in concentration inside the chamber.

Set the 'limit.pt' based on the number of standard errors above the mean that will determine 'outliers'.

```
hist(coefs$slope_se)
```



```
boxplot(coefs$slope_se)
```



```

limit.pt<- mean(coefs$slope_se) + (sd(coefs$slope_se) * 2) #change the final number based on preference

#filter out data that does not meet the above limit.pt criteria
coefs.clean <- coefs %>%
  filter(slope_se < limit.pt)

#record which slopes did not meet criteria
coefs.out <- coefs %>%
  filter(slope_se > limit.pt)

#create a reference dataframe that excludes data from the codes just determined to meet criteria for removal
mydat.clean<- mydat %>%
  filter(!code %in% coefs.out$code)

#check the correct amount of codes were removed
length(unique(mydat$code)) #this number...

## [1] 2880

length(unique(coefs.out$code)) #minus this number...

## [1] 28

length(unique(mydat.clean$code)) #should equal this number

## [1] 2852

```


#plot this data

mydat.clean %>%

```
ggplot(aes(count, co2_ppm, color = as.factor(plot))) +
  geom_point() +
  geom_line() +
  facet_wrap(~date)
```



mydat.clean %>%

```
ggplot(aes(count, co2_ppm, color = as.factor(date))) +
  geom_point() +
  geom_line() +
  facet_wrap(~plot)
```

2020-11-02		2022-06-01		2022-10-11		2023-06-06		2023-07-25		2023-10-20	
2020-11-05		2022-06-07		2022-10-21		2023-06-09		2023-08-01		2023-10-23	
2020-11-19		2022-06-08		2022-10-27		2023-06-14		2023-08-02		2023-10-27	
2021-04-05		2022-06-16		2023-04-13		2023-06-15		2023-08-04		2023-11-07	
2021-04-22		2022-06-28		2023-04-14		2023-06-19		2023-08-07		2024-03-13	
2021-04-29		2022-07-01		2023-04-25		2023-06-20		2023-08-08		2024-03-19	
2021-05-18		2022-07-14		2023-04-26		2023-06-21		2023-08-15		2024-04-04	
2021-06-01		2022-07-21		2023-05-08		2023-06-27		2023-08-16		2024-04-09	
2021-06-15		2022-08-01		2023-05-09		2023-06-28		2023-08-17		2024-04-15	
2021-06-21		2022-08-03		2023-05-10		2023-07-03		2023-08-21		2024-04-22	
2021-07-08		2022-08-31		2023-05-16		2023-07-04		2023-08-30		2024-04-25	
2021-07-16		2022-09-02		2023-05-17		2023-07-05		2023-09-12		2024-05-01	
2021-08-10		2022-09-08		2023-05-22		2023-07-06		2023-09-13		2024-05-03	
2021-09-10		2022-09-15		2023-05-23		2023-07-10		2023-09-14		2024-05-06	
2021-10-14		2022-09-19		2023-05-24		2023-07-11		2023-09-26		2024-05-09	
2021-10-25		2022-09-21		2023-05-25		2023-07-17		2023-09-27		2024-05-22	
2022-05-04		2022-09-27		2023-05-31		2023-07-19		2023-10-02		2024-05-23	
2022-05-05		2022-10-03		2023-06-01		2023-07-20		2023-10-09		2024-05-29	
2022-05-16		2022-10-07		2023-06-05		2023-07-24		2023-10-18		2024-05-30	

OPTIONAL: Determine Significance of Slope

You may choose to force to zero the slopes that are not significantly different from zero. To do that, run an anova on all regressions and extract the codes for the slopes with p-values greater than 0.05. Then, make those slopes zero. Store this data in a relevant folder.

```
#Rows with 1 degree of freedom will be the slopes. This way we don't also get the residuals.
#pvals<-ldply(regressions, anova)
#pvals2<-subset(pvals, Df=="1")

# Create a table with codes (i.e. plots) where slope WAS NOT significant.
#pvals3<-subset(pvals2, pvals2[,6]>0.05)

#Find the slopes in the coefs.sum dataframe using these codes from pvals3 (which are not significant) a
#coefs.sum[which(coefs.sum$code %in% pvals3$code),c(3,5)]<-0
#head(coefs.sum)
```

If not forcing to zero, rename dataframe.

```
coefs.sum <- coefs.clean
```

Convert Slopes to Fluxes

Create a dataframe with the dimensions of the chamber used. These dimensions are based on calculations done by Bergquist, Jungers, and Gutknecht on the chamber they designed.

Convert slope value to CO2 equivalents. Then use chamber height to remove 3rd dimension—leaving a value of rate of CO2 respiration per unit area (ug/cm2/s). Two conversions are made to relevant units, including g/m2/day and kg/ha/yr.

Export final flux values to a relevant folder.

```
chamberdims<-data.frame(matrix(ncol=5))
colnames(chamberdims)<-c("clength", "cwidth","carea","cvol","cheight")
chamberdims$clength<-50.165 #cm
chamberdims$cwidth<-13.97
chamberdims$carea<-chamberdims$clength*chamberdims$cwidth
chamberdims$cvol<-6250 #cm3. Known by filling chamber with water.
chamberdims$cheight<-chamberdims$cvol/chamberdims$carea

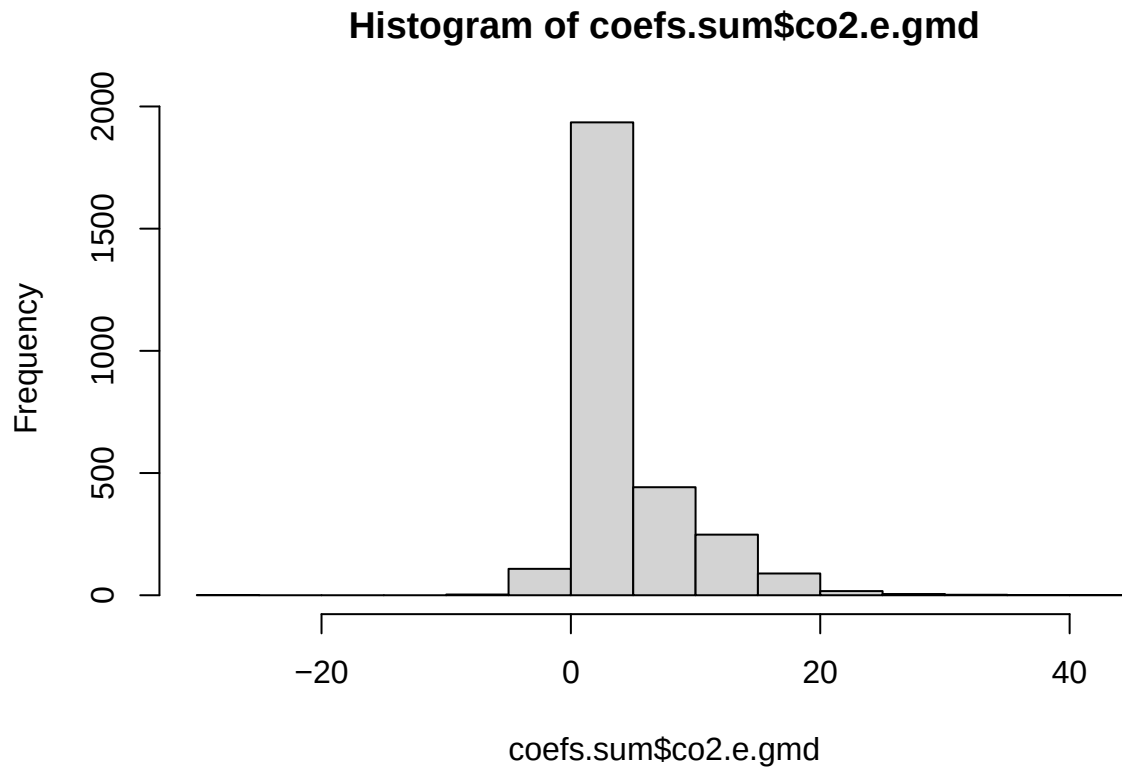
#converting co2.c to co2 equivalents.
c.to.e <- function(x) {
  x * (44/12)
}

#converts ug/cm2/s to g/m2/day. 1 g = 1,000,000 ug. 10,000 cm2 = 1 m2. 86,400 sec = 1 day.
area.to.gmd <- function(x) {
  x * (1/1000000) * (10000/1) * (86400/1)
}

#converts ug/cm2/s to kg/ha/year. 1 kg = 1,000,000,000 ug. 100,000,000 cm2 = 1 Ha. 31,536,000 sec = 1 y
area.to.kghyr <- function(x) {
  x * (1/1e9) * (1e8/1) * (3.1536e7/1)
}

coefs.sum <- coefs.sum %>%
  dplyr::mutate(co2.e = c.to.e(slope),
    co2.e.area = co2.e * chamberdims$cheight,
    co2.e.gmd = area.to.gmd(co2.e.area),
    co2.e.kghyr = area.to.kghyr(co2.e.area),
    day = as.numeric(strftime(date, format="%j")))

#review the distribution of the final flux values
hist(coefs.sum$co2.e.gmd)
```



Review Removed Data

Compare counts of the number of plots sampled per day and number of dates for each plot in both raw data (pre) and final data (post). Be sure there aren't any dates or plots that have disproportionately high data removal.

```
pre.date <- dat.time %>%
  select(plot, date) %>%
  unique() %>%
  group_by(date) %>%
  tally() %>%
  mutate(date = as.character(date))

## Adding missing grouping variables: `code`

post.date <- coefs.sum %>%
  select(plot, date) %>%
  unique() %>%
  group_by(date) %>%
  tally()

left_join(pre.date, post.date, by = 'date', suffix = c('.pre', '.post'))

## # A tibble: 149 x 3
##   date      n.pre n.post
##   <chr>    <int> <int>
## 1 2020-11-02    24     23
```

```
## 2 2020-11-05    24    22
## 3 2020-11-19    24    24
## 4 2021-04-05    24    22
## 5 2021-04-22    21    20
## 6 2021-04-29    24    23
## 7 2021-05-18    24    20
## 8 2021-06-01    23    22
## 9 2021-06-15    24    23
## 10 2021-06-21     1     1
## # i 139 more rows
```

```
pre.plot <- dat.time %>%
  select(plot, date) %>%
  unique() %>%
  group_by(plot) %>%
  tally() %>%
  mutate(plot = as.character(plot))
```

Adding missing grouping variables: `code`

```
post.plot <- coefs.sum %>%
  select(plot, date) %>%
  unique() %>%
  group_by(plot) %>%
  tally()
```

```
left_join(pre.plot, post.plot, by = 'plot', suffix = c('.pre', '.post'))
```

```
## # A tibble: 182 x 3
##   plot n.pre `n,post`
##   <chr> <int>   <int>
## 1 1      30      27
## 2 101    16      15
## 3 102    16      14
## 4 103    16      15
## 5 104    15      15
## 6 105     1       1
## 7 106     1       1
## 8 107    16      12
## 9 108    15      15
## 10 109     1       1
## # i 172 more rows
```

Saving Intermediary Data

In a relevant folder, save the following dataframes as .csv files for future reference.

Jesse saving CO2 data

```
coefs.sum %>%
  select(experiment, site, plot, date, co2.e.kghyr) %>%
  write.csv("kgml_CO2_v1.csv")

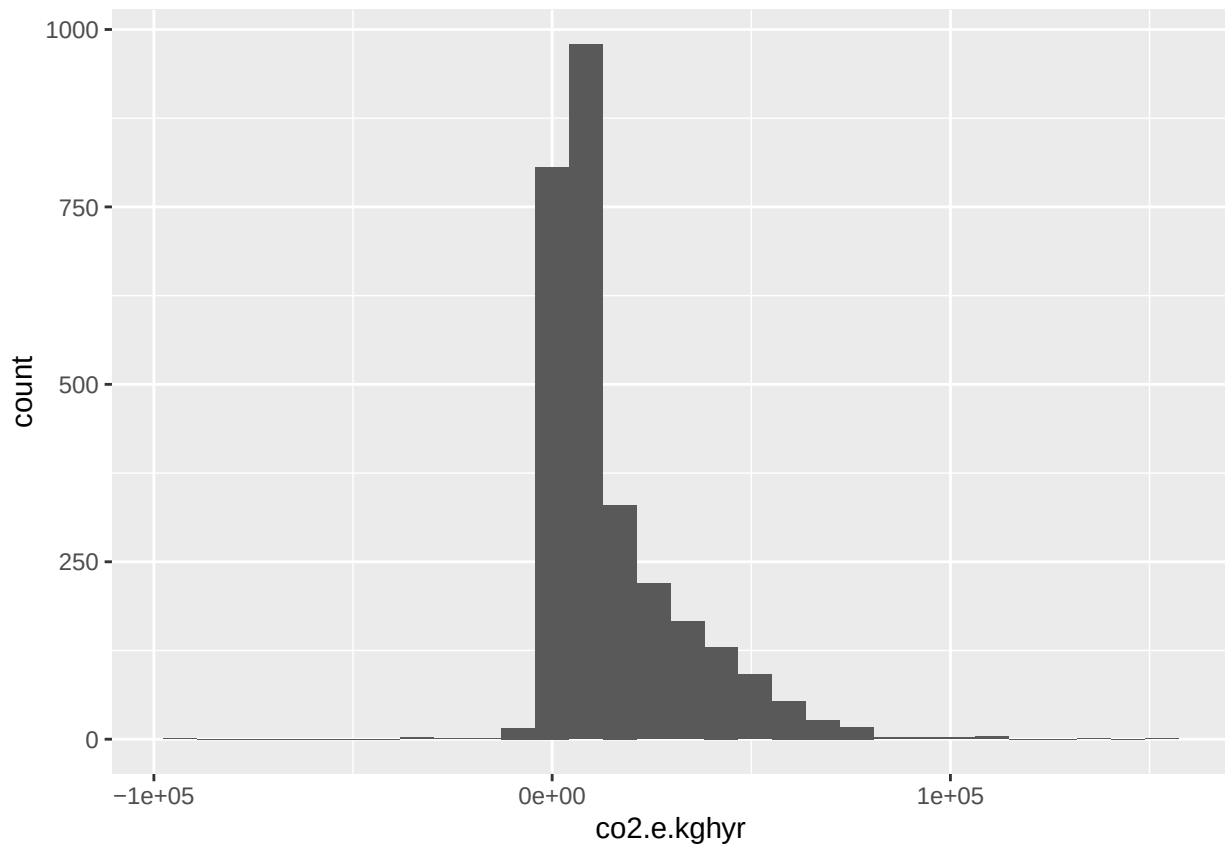
# checking CO2 data
```

```

coefs.sum %>%
  # filter(co2.e.kghyr>-10000 &
    # co2.e.kghyr<100000) %>%
  ggplot(aes(co2.e.kghyr)) +
  geom_histogram()

```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



N2O

Converting n2o concentrations (ppm n2o) to ug N/cm3.

The equation is $=(\text{element mass/ambient pressure}) * (1/1000000) * (1/0.082057) * (1/273.15 + \text{chamber temp}) * 1000$

Then create a new dataframe, which selects a subset of columns, creates a unique ID column 'code', and uses the created function to convert to co2.c using co2 and cell temp columns.

Then, save this intermediate data in a relevant folder on your computer.

```

n2o.convert <- function(n2o, temp) {
  n2o*(14/1)*(1/1000000)*(1/0.082057)*(1/(273.15 + temp)) * 1000
}

mydat<- dat.slim %>%
  dplyr::select(code, date, plot, experiment, site, datetime, co2_ppm, n2o_ppm, run.t, temp_celsius, co

```

```
dplyr::mutate(co2.c = co2.convert(co2_ppm, temp_celsius)) %>%
dplyr::mutate(n2o.c = n2o.convert(n2o_ppm, temp_celsius))
```

Calculating Slopes

Create a list of regressions for every ‘code’ (i.e., plot/date/field combination). Then create a dataframe (“coefs”) that includes the intercepts and slopes of each model. Then, separate “code” column, so plot, date, and site are in separate columns. Rename columns. Store this data in a relevant folder.

```
regressions <- dplyr::group_by(mydat, .(code), lm, formula = n2o.c ~ run.t)

coefs <- cbind(ldply(regressions, coef), #extracts the code variables of each model, and the 5th coefficient
              ldply(regressions, function(x){coef(summary(x))[,2]}), #extracts the standard error
              function(x){coef(summary(x))[,2:3]}))

str_split_fixed(coefs$code, "_", n=3)[,1] -> coefs$experiment
str_split_fixed(coefs$code, "_", n=3)[,2] -> coefs$plot
str_split_fixed(coefs$code, "_", n=4)[,3] -> coefs$date
str_split_fixed(coefs$code, "_", n=4)[,4] -> coefs$site

colnames(coefs)<-c("code", "intercept", "slope", "intercept_se", "slope_se", "experiment", "plot", "date", "site")
```

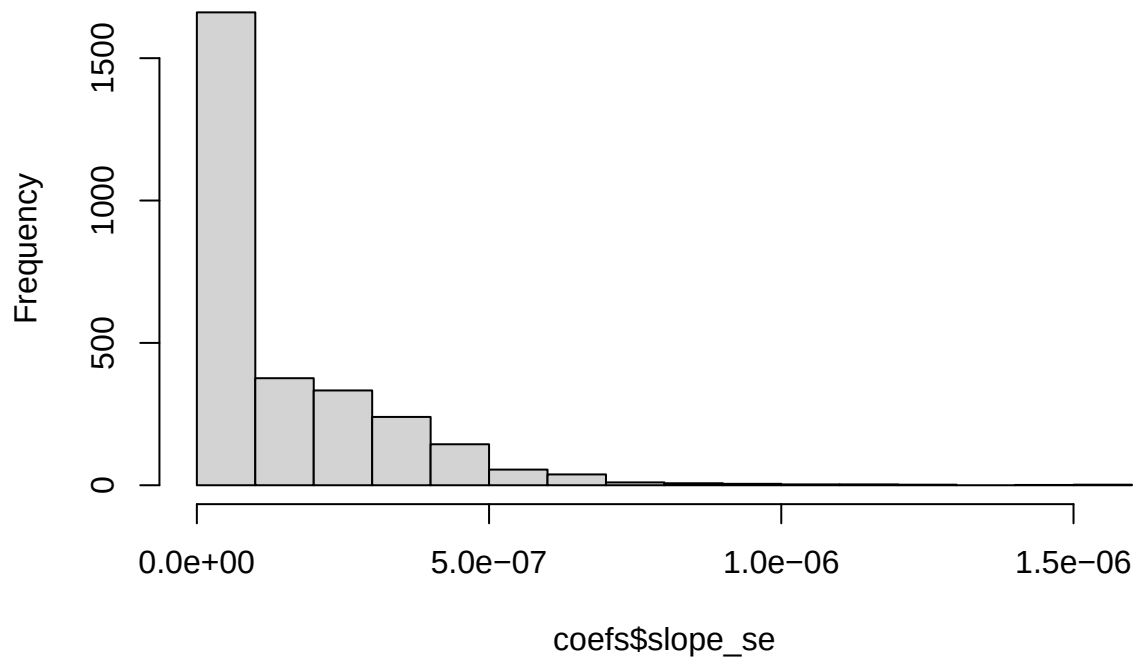
OPTIONAL: Remove low-quality slopes based on standard error

In order to remove poor quality data, the slope standard error could be a criteria for removal. Typically, poor data comes from an erratic high/low concentration measurement that falls beyond the assumed linear increase in concentration inside the chamber.

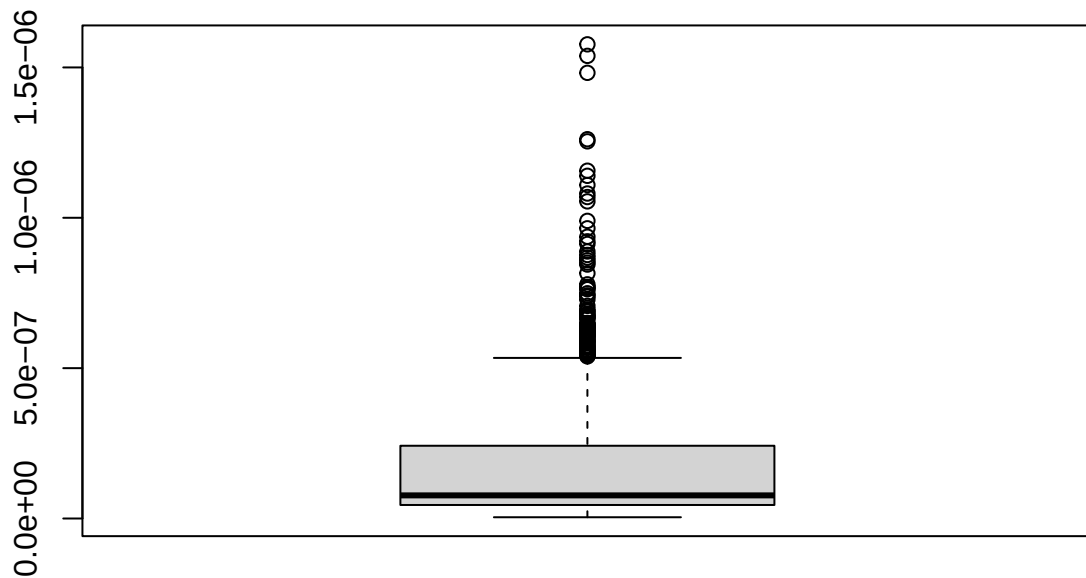
Set the ‘limit.pt’ based on the number of standard errors above the mean that will determine ‘outliers’.

```
hist(coefs$slope_se)
```

Histogram of coefs\$slope_se



```
boxplot(coefs$slope_se)
```

```

limit.pt<- mean(coefs$slope_se) + (sd(coefs$slope_se) * 2) #change the final number based on preference

#filter out data that does not meet the above limit.pt criteria
coefs.clean <- coefs %>%
  filter(slope_se < limit.pt)

#record which slopes did not meet criteria
coefs.out <- coefs %>%
  filter(slope_se > limit.pt)

#create a reference dataframe that exludes data from the codes just determined to meet criteria for rem
mydat.clean<- mydat %>%
  filter(!code %in% coefs.out$code)

#check the correct amount of codes were removed
length(unique(mydat$code)) #this number...

## [1] 2880

length(unique(coefs.out$code)) #minus this number...

## [1] 126

length(unique(mydat.clean$code)) #should equal this number

## [1] 2754

```

#plot this data

mydat.clean %>%

```
ggplot(aes(count, co2_ppm, color = as.factor(plot))) +
  geom_point() +
  geom_line() +
  facet_wrap(~date)
```



mydat.clean %>%

```
ggplot(aes(count, co2_ppm, color = as.factor(date))) +
  geom_point() +
  geom_line() +
  facet_wrap(~plot)
```

2020-11-02		2022-06-01		2022-10-11		2023-06-06		2023-07-25		2023-10-20	
2020-11-05		2022-06-07		2022-10-21		2023-06-09		2023-08-01		2023-10-23	
2020-11-19		2022-06-08		2022-10-27		2023-06-14		2023-08-02		2023-10-27	
2021-04-05		2022-06-16		2023-04-13		2023-06-15		2023-08-04		2023-11-07	
2021-04-22		2022-06-28		2023-04-14		2023-06-19		2023-08-07		2024-03-13	
2021-04-29		2022-07-01		2023-04-25		2023-06-20		2023-08-08		2024-03-19	
2021-05-18		2022-07-14		2023-04-26		2023-06-21		2023-08-15		2024-04-04	
2021-06-01		2022-07-21		2023-05-08		2023-06-27		2023-08-16		2024-04-09	
2021-06-15		2022-08-01		2023-05-09		2023-06-28		2023-08-17		2024-04-15	
2021-06-21		2022-08-03		2023-05-10		2023-07-03		2023-08-21		2024-04-22	
2021-07-08		2022-08-31		2023-05-16		2023-07-04		2023-08-30		2024-04-25	
2021-07-16		2022-09-02		2023-05-17		2023-07-05		2023-09-12		2024-05-01	
2021-08-10		2022-09-08		2023-05-22		2023-07-06		2023-09-13		2024-05-03	
2021-09-10		2022-09-15		2023-05-23		2023-07-10		2023-09-14		2024-05-06	
2021-10-14		2022-09-19		2023-05-24		2023-07-11		2023-09-26		2024-05-09	
2021-10-25		2022-09-21		2023-05-25		2023-07-17		2023-09-27		2024-05-22	
2022-05-04		2022-09-27		2023-05-31		2023-07-19		2023-10-02		2024-05-23	
2022-05-05		2022-10-03		2023-06-01		2023-07-20		2023-10-09		2024-05-29	
2022-05-16		2022-10-07		2023-06-05		2023-07-24		2023-10-18		2024-05-30	

OPTIONAL: Determine Significance of Slope

You may choose to force to zero the slopes that are not significantly different from zero. To do that, run an anova on all regressions and extract the codes for the slopes with p-values greater than 0.05. Then, make those slopes zero. Store this data in a relevant folder.

```
#Rows with 1 degree of freedom will be the slopes. This way we don't also get the residuals.
#pvals<-ldply(regressions, anova)
#pvals2<-subset(pvals, Df=="1")
```

```
# Create a table with codes (i.e. plots) where slope WAS NOT significant.
#pvals3<-subset(pvals2, pvals2[,6]>0.05)
```

```
#Find the slopes in the coefs.sum dataframe using these codes from pvals3 (which are not significant) a
#coefs.sum[which(coefs.sum$code %in% pvals3$code),c(3,5)]<-0
#head(coefs.sum)
```

If not forcing to zero, rename dataframe.

```
coefs.sum <- coefs.clean
```

Convert Slopes to Fluxes

Create a dataframe with the dimensions of the chamber used. These dimensions are based on calculations done by Bergquist, Jungers, and Gutknecht on the chamber they designed.

Convert slope value to CO₂ equivalents. Then use chamber height to remove 3rd dimension—leaving a value of rate of CO₂ respiration per unit area (ug/cm²/s). Two conversions are made to relevant units, including g/m²/day and kg/ha/yr.

Export final flux values to a relevant folder.

```
chamberdims<-data.frame(matrix(ncol=5))
colnames(chamberdims)<-c("clength", "cwidth","carea","cvol","cheight")
chamberdims$clength<-50.165 #cm
chamberdims$cwidth<-13.97
chamberdims$carea<-chamberdims$clength*chamberdims$cwidth
chamberdims$cvol<-6250 #cm3. Known by filling chamber with water.
chamberdims$cheight<-chamberdims$cvol/chamberdims$carea

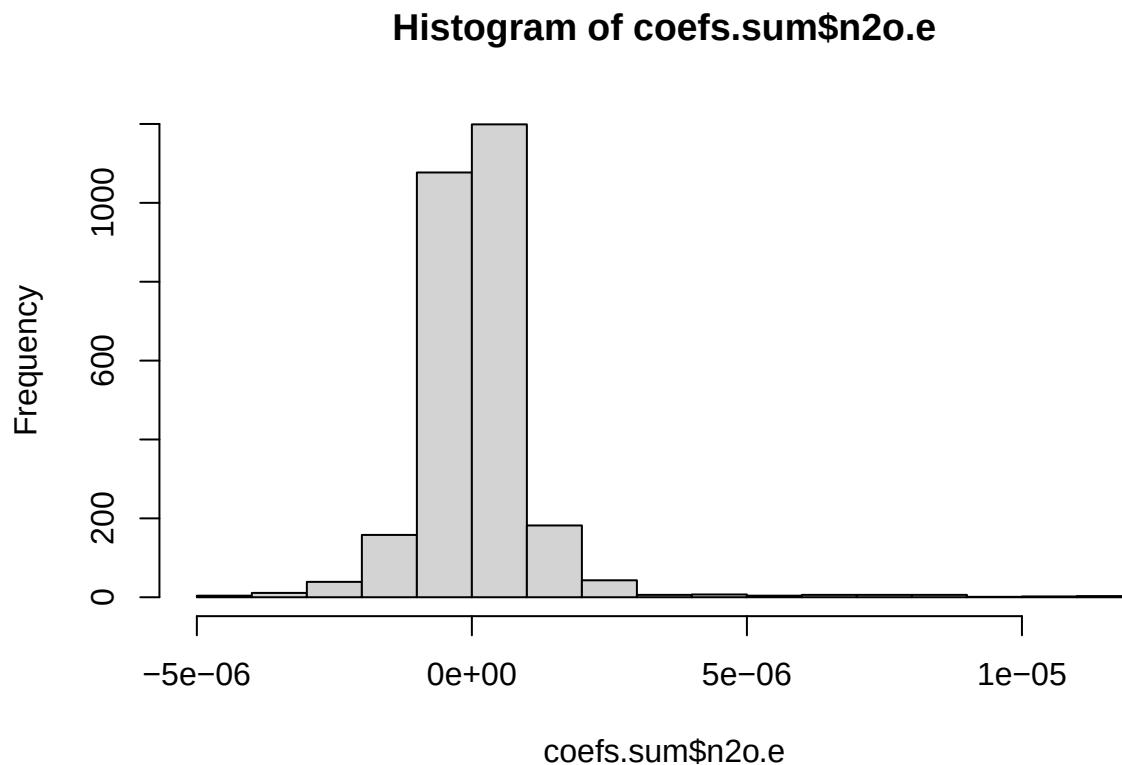
#converting co2.c to co2 equivalents.
c.to.e <- function(x) {
  x * (44/12)
}

#converts ug/cm2/s to g/m2/day. 1 g = 1,000,000 ug. 10,000 cm2 = 1 m2. 86,400 sec = 1 day.
area.to.gmd <- function(x) {
  x * (1/1000000) * (10000/1) * (86400/1)
}

#converts ug/cm2/s to kg/ha/year. 1 kg = 1,000,000,000 ug. 100,000,000 cm2 = 1 Ha. 31,536,000 sec = 1 y
area.to.kghyr <- function(x) {
  x * (1/1e9) * (1e8/1) * (3.1536e7/1)
}

coefs.sum <- coefs.sum %>%
  dplyr::mutate(co2.e = c.to.e(slope),
    co2.e.area = co2.e * chamberdims$cheight,
    co2.e.gmd = area.to.gmd(co2.e.area),
    co2.e.kghyr = area.to.kghyr(co2.e.area),
    n2o.e = c.to.e(slope),
    day = as.numeric(strftime(date, format="%j")))

#review the distribution of the final flux values
hist(coefs.sum$n2o.e)
```



Review Removed Data

Compare counts of the number of plots sampled per day and number of dates for each plot in both raw data (pre) and final data (post). Be sure there aren't any dates or plots that have disproportionately high data removal.

```
pre.date <- dat.time %>%
  select(plot, date) %>%
  unique() %>%
  group_by(date) %>%
  tally() %>%
  mutate(date = as.character(date))

## Adding missing grouping variables: `code`

post.date <- coefs.sum %>%
  select(plot, date) %>%
  unique() %>%
  group_by(date) %>%
  tally()

left_join(pre.date, post.date, by = 'date', suffix = c('.pre', '.post'))

## # A tibble: 149 x 3
##   date      n.pre n.post
##   <chr>    <int> <int>
## 1 2020-11-02    24     23
```

```
## 2 2020-11-05    24    21
## 3 2020-11-19    24    24
## 4 2021-04-05    24    22
## 5 2021-04-22    21    20
## 6 2021-04-29    24    23
## 7 2021-05-18    24    20
## 8 2021-06-01    23    22
## 9 2021-06-15    24    23
## 10 2021-06-21     1     1
## # i 139 more rows
```

```
pre.plot <- dat.time %>%
  select(plot, date) %>%
  unique() %>%
  group_by(plot) %>%
  tally() %>%
  mutate(plot = as.character(plot))
```

Adding missing grouping variables: `code`

```
post.plot <- coefs.sum %>%
  select(plot, date) %>%
  unique() %>%
  group_by(plot) %>%
  tally()
```

```
left_join(pre.plot, post.plot, by = 'plot', suffix = c('.pre', '.post'))
```

```
## # A tibble: 182 x 3
##   plot n.pre `n,post`
##   <chr> <int>   <int>
## 1 1      30      29
## 2 101    16      16
## 3 102    16      15
## 4 103    16      15
## 5 104    15      15
## 6 105     1       1
## 7 106     1       1
## 8 107    16      13
## 9 108    15      15
## 10 109     1       1
## # i 172 more rows
```

Saving Intermediary Data

In a relevant folder, save the following dataframes as .csv files for future reference.

Jesse saving CO2 data

```
coefs.sum %>%
  select(experiment, site, plot, date, co2.e.kghyr) %>%
  write.csv("kgml_CO2_v1.csv")

# checking CO2 data
```

```
coefs.sum %>%  
  # filter(co2.e.kghyr>-10000 &  
    # co2.e.kghyr<100000) %>%  
  ggplot(aes(co2.e.kghyr)) +  
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

